Automação de Atendimento via WhatsApp: Documento de Requisitos e Especificações

Versão 1.0 · Chatbot de atendimento com WhatsApp Cloud API, handoff humano e IA restrita ao escopo do negócio · Texto revisado em 08/11/2026

1. Visão Geral

1.1 Objetivo

Construir um sistema de atendimento automatizado via WhatsApp que receba mensagens de clientes, responda perguntas frequentes, consulte status de pedidos, realize agendamentos e encaminhe para um atendente humano quando necessário. O projeto é desenhado para ser **testável por você desde o primeiro dia, sem custo e sem arriscar um número real**.

1.2 Problema

Pequenas empresas atendem clientes pelo WhatsApp de forma manual: demora na resposta, perguntas repetidas, perda de histórico e nenhum controle de qualidade. Automatizar o repetitivo e escalar o complexo para pessoas melhora tempo de resposta e libera a equipe.

1.3 Objetivos mensuráveis

ID	Objetivo	Meta
OBJ-01	Tempo da primeira resposta	menor que 5 s (p95)
OBJ-02	Perguntas resolvidas sem humano	maior ou igual a 60% no conjunto de diálogos de teste
OBJ-03	Encaminhamento correto ao humano quando solicitado ou necessário	maior ou igual a 95%
OBJ-04	Mensagens duplicadas processadas	0
OBJ-05	Custo para desenvolver e testar	R$ 0

1.4 Domínio de exemplo

Para tornar o projeto concreto, o documento usa uma **clínica de pequeno porte / loja de serviços** (horários, preços, agendamento, status de pedido). O domínio é configurável: basta trocar a base de conhecimento e os fluxos.

2. Verificação das Informações sobre Canais e Custos

O texto de referência contém pontos corretos e outros que precisam de ajuste. Conferi fontes atuais; **confirme sempre os valores na página oficial de preços da Meta antes de ir à produção**, pois mudam com frequência.

#	Afirmação do texto	Situação verificada	Impacto no projeto
V1	Há 1.000 mensagens de serviço gratuitas por mês por número	Em linha com o modelo atual: respostas na janela de 24 h foram gratuitas de nov/2024 a set/2026; desde 01/10/2026 há 1.000 gratuitas por número/mês e, depois disso, cobrança por mensagem no valor de utility do país. Fontes divergem em detalhes e algumas páginas antigas ainda dizem gratuito e ilimitado	Usar contador de mensagens e alerta de consumo (RF-18)
V2	Custo de cerca de US$ 0,0068 por resposta no Brasil	Não consegui confirmar o valor para o Brasil; a faixa citada em fontes é de utility templates e muda por país e data	Tratar como estimativa; consultar a tabela oficial de tarifas
V3	Modelo de cobrança por conversa	Substituído por cobrança **por mensagem** desde 01/07/2025	Registrar categoria de cada mensagem enviada

#	Afirmação do texto	Situação verificada	Impacto no projeto
V4	Mensagens ativas exigem templates aprovados	Correto: fora da janela de 24 h só é possível enviar templates aprovados (marketing, utility, autenticação)	Regra RN-02
V5	Número de teste gratuito com destinatários verificados	Correto: ambiente de teste com número fornecido pela Meta e lista limitada de destinatários verificados (confirmar o limite vigente no painel)	Base da estratégia de teste (seção 3)
V6	Risco de banimento praticamente zero na API oficial	Exagero: a API oficial também pode ter número restrito por violação de política, baixa qualidade, falta de opt-in ou spam	Regras de opt-in, qualidade e monitoramento
V7	Não há política sobre IA	**Há:** desde 15/01/2026 os Termos proíbem provedores de IA de oferecer assistentes de uso geral pela plataforma. Bots de suporte com escopo restrito ao negócio continuam permitidos	IA obrigatoriamente restrita ao escopo (RN-06)
V8	Evolution API/Baileys/WPPConnect sem custo	Verdadeiro, mas viola os termos da Meta e arrisca o número	Apenas adaptador experimental opcional (seção 24)
V9	Exige cartão de crédito	Para uso em produção é necessário método de pagamento no negócio; para o teste inicial não precisei confirmar exigência. Verificar no seu painel	Não bloqueia o desenvolvimento

Decisão: usar a **WhatsApp Cloud API oficial com o número de teste da Meta** como canal principal, por ser gratuita, estável e reproduzir o comportamento real (janela de 24 h, webhooks, templates). O sistema usa uma camada de abstração de canal, o que permite um simulador local para testes automatizados.

3. Estratégia para Testar e Funcionar (3 ambientes)

Ambiente	Canal	Quando usar	Custo
Local/CI	Simulador próprio de WhatsApp (envia payloads de webhook gravados e captura as respostas)	Testes automatizados, desenvolvimento rápido, sem internet	R$ 0
Sandbox Meta	Cloud API com número de teste e destinatários verificados, webhook exposto por túnel HTTPS (cloudflared ou ngrok)	Testes ponta a ponta com seu celular	R$ 0
Produção	Cloud API com número próprio verificado e templates aprovados	Cliente real	1.000 mensagens de serviço gratuitas/número/mês, depois conforme tabela da Meta

Regra de ouro: nenhuma regra de negócio depende diretamente da API da Meta. Tudo passa pela interface `ChannelAdapter` (`parse_inbound` , `send_text` , `send_interactive` , `send_template` , `mark_read`). Assim você testa 95% do sistema sem WhatsApp.

4. Escopo

4.1 Dentro do escopo (MVP)

- Recebimento de mensagens por webhook, com verificação de assinatura e deduplicação.
- Menu inicial com botões e listas interativas.

- FAQ com base de conhecimento do negócio (busca simples e depois RAG).
- Consulta de status de pedido e agendamento (integrações simuladas por API interna).
- Handoff para atendente humano com fila e painel mínimo.
- Controle da janela de 24 h e envio de templates fora dela.
- Registro de consentimento (opt-in/opt-out), histórico e métricas.
- Camada de IA (opcional, fase 2) restrita ao escopo do negócio.

4.2 Fora do escopo (versão 1)

- Disparo em massa de marketing.
- Pagamentos dentro do WhatsApp.
- Multi-tenant (várias empresas) e cobrança do serviço.
- Voz, chamadas e grupos.

4.3 Evolução futura

Painel web completo para atendentes, integração com CRM/ERP reais, múltiplos números, análise de sentimento, relatórios de satisfação (CSAT).

5. Personas

Persona	Necessidade
Cliente final	Resposta rápida a dúvidas, agendar e acompanhar pedido sem esperar
Atendente	Receber só o que a automação não resolve, com contexto da conversa
Gestor	Ver volume, tempo de resposta, taxa de resolução e custo
Desenvolvedor (você)	Ambiente reproduzível, testável e sem custo

6. Jornadas de Conversa

Jornada	Fluxo resumido
Primeiro contato	Saudação, aviso de uso de dados e do bot, pedido de consentimento, menu principal

Jornada	Fluxo resumido
FAQ	Pergunta livre, busca na base, resposta com fonte, pergunta de confirmação (resolveu?)
Status de pedido	Pede número do pedido, valida formato, consulta, responde com status e previsão
Agendamento	Escolhe serviço, data e horário (lista), confirma, envia comprovante e lembrete
Falar com humano	Reconhece o pedido (botão ou texto), avisa horário de atendimento, cria ticket e passa o controle
Fora de escopo	Informa que só atende assuntos do negócio e reapresenta o menu
Encerramento	Pesquisa rápida de satisfação (nota de 1 a 5)
Opt-out	Cliente digita PARAR; sistema confirma e interrompe mensagens ativas

7. Requisitos Funcionais

Prioridade MoSCoW: **M** (must), **S** (should), **C** (could).

ID	Requisito	Prior.
RF-01	Expor `GET /webhook` para a verificação da Meta (retornar o `hub.challenge` quando o token confere)	M
RF-02	Expor `POST /webhook` que valida a assinatura `X-Hub-Signature-256` com o segredo do app e responde 200 rapidamente	M
RF-03	Processar mensagens de forma assíncrona (fila) após responder 200	M
RF-04	Deduplicar eventos pelo identificador da mensagem (`wamid`), pois a Meta reenvia em caso de falha	M
RF-05	Suportar mensagens recebidas de texto, botão, lista e mídia (imagem/áudio/documento) com resposta apropriada	M

ID	Requisito	Prior.
RF-06	Manter sessão por cliente com estado da conversa (máquina de estados) e expiração configurável	M
RF-07	Enviar menus interativos (botões e listas) e mensagens de texto	M
RF-08	Marcar mensagens como lidas e exibir indicador de digitação quando disponível	S
RF-09	Responder FAQ a partir da base de conhecimento, citando a fonte (título do artigo)	M
RF-10	Consultar status de pedido por número validado	M
RF-11	Criar, consultar e cancelar agendamentos com verificação de disponibilidade	S
RF-12	Realizar handoff para humano com ticket, resumo da conversa e fila	M
RF-13	Painel mínimo para o atendente ver conversas e responder (pode ser a interface do Chatwoot ou uma tela própria simples)	S
RF-14	Controlar a janela de 24 h e usar templates aprovados fora dela	M
RF-15	Registrar opt-in, opt-out e histórico de consentimento	M
RF-16	Horário de atendimento e mensagem automática fora do expediente	S
RF-17	Camada de IA para classificar intenção e responder dúvidas, restrita ao escopo (fase 2)	S
RF-18	Contabilizar mensagens enviadas por categoria e alertar ao se aproximar da cota gratuita	S
RF-19	Pesquisa de satisfação ao final do atendimento	C
RF-20	Exportar métricas e conversas anonimizadas	C
RF-21	Fornecer um simulador de WhatsApp para testes locais	M
RF-22	Expor `GET /health` e `GET /metrics`	S

8. Requisitos Não Funcionais

ID	Categoria	Requisito
RNF-01	Desempenho	ACK do webhook em menos de 1 s (p95); primeira resposta em menos de 5 s (p95)
RNF-02	Confiabilidade	Retentativas com backoff para envios; nenhuma mensagem perdida em reinício; processamento idempotente
RNF-03	Disponibilidade	Reinício automático dos serviços; metas de 99% em produção pequena
RNF-04	Segurança	Validação de assinatura, segredos em variáveis de ambiente, tokens de acesso permanentes de usuário do sistema, rate limiting
RNF-05	Privacidade	LGPD: minimização de dados, mascaramento de telefone em logs, política de retenção, exclusão sob demanda
RNF-06	Observabilidade	Logs estruturados com `conversation_id`, métricas de latência, taxa de handoff, erros de envio
RNF-07	Portabilidade	`docker compose up` sobe API, worker, banco e Redis
RNF-08	Testabilidade	Simulador e testes de diálogo; cobertura maior ou igual a 80% na lógica de fluxo
RNF-09	Custo	Uso de ferramentas gratuitas e controle de consumo de mensagens
RNF-10	Manutenibilidade	Camada de canal abstrata; versão da Graph API configurável por variável

9. Regras de Negócio

- **RN-01 (janela de 24 h):** mensagens livres só podem ser enviadas até 24 h após a última mensagem do cliente.
- **RN-02 (templates):** fora da janela, apenas templates aprovados; templates de marketing exigem opt-in explícito.
- **RN-03 (opt-in/opt-out):** nenhuma mensagem ativa sem consentimento; palavras como PARAR, SAIR e CANCELAR encerram imediatamente o envio ativo.

- **RN-04 (humano sempre acessível):** o cliente pode pedir atendente a qualquer momento; o bot não bloqueia esse pedido (também exigido por regras de SAC no Brasil; verificar o texto vigente).
- **RN-05 (identificação do bot):** a primeira mensagem informa que é um atendimento automatizado e como falar com uma pessoa.
- **RN-06 (escopo da IA):** a IA responde apenas sobre o negócio (serviços, preços, horários, pedidos, agendamentos). Assuntos gerais são recusados com mensagem padrão. Isso mantém o produto como bot de suporte e não como assistente de uso geral, conforme os Termos da Meta.
- **RN-07 (fundamentação):** respostas de FAQ vêm da base de conhecimento; sem base, o bot admite que não sabe e oferece o humano.
- **RN-08 (dados sensíveis):** o bot não solicita senhas, números completos de cartão ou dados de saúde além do estritamente necessário ao agendamento.
- **RN-09 (limite de tentativas):** após 2 falhas de entendimento seguidas, oferecer humano.
- **RN-10 (idempotência):** cada `wamid` é processado uma única vez.
- **RN-11 (qualidade):** monitorar taxa de bloqueios e reclamações; pausar envios ativos se a qualidade do número cair.

10. Arquitetura

10.1 Fluxo principal

1. Cliente envia mensagem pelo WhatsApp.
2. A Meta chama `POST /webhook` da sua API.
3. A API valida assinatura, grava o evento bruto, deduplica e enfileira; responde 200.
4. O worker consome a fila, carrega a sessão e executa a máquina de estados (e a IA, quando habilitada).
5. O worker envia a resposta pelo `ChannelAdapter`.
6. Status de entrega (enviado, entregue, lido, falha) chegam por webhook e atualizam o registro.
7. Handoff cria ticket e notifica o painel do atendente.

10.2 Stack recomendada (gratuita)

Camada	Tecnologia	Observação
Linguagem/API	Python 3.12 + FastAPI	Webhook assíncrono e documentação OpenAPI
Fila e sessão	Redis + Celery (ou RQ/arq)	Deduplicação com chaves de expiração
Banco	PostgreSQL 16	Conversas, tickets, consentimentos
Canal	WhatsApp Cloud API (Graph API)	Versão configurável
Túnel de desenvolvimento	cloudflared ou ngrok (plano gratuito)	Meta exige URL HTTPS pública
IA (fase 2)	Ollama local ou Groq (tier gratuito) com RAG simples sobre a base de conhecimento	Abstraído por interface; guardrails da seção 13
Atendimento humano	Chatwoot (open source) ou painel próprio	Chatwoot reduz muito o esforço
Testes	pytest, respx/httpx mock, simulador próprio	
Infra	Docker Compose	api, worker, db, redis, simulador

10.3 Estrutura do repositório

```
whatsapp-atendimento/
  docker/
  src/
    api/ (webhook, rotas internas)
    channel/ (adapter.py, meta_cloud.py, simulator.py)
    flows/ (máquina de estados, menus, templates de resposta)
    nlu/ (regras, classificador, guardrails)
    knowledge/ (base de FAQ, busca/RAG)
    integrations/ (pedidos, agenda - mocks e reais)
    handoff/ (tickets e fila)
    consent/ (opt-in e opt-out)
    db/ (modelos e migrações)
  tests/ (unit, dialogs, contract)
  simulator/ (CLI e payloads gravados)
  docs/
```

11. Integração com a Cloud API: Passo a Passo

1. Criar conta no Meta for Developers e um app do tipo Business.
2. Adicionar o produto WhatsApp; o painel fornece um **número de teste**, o identificador do número (`phone_number_id`) e um token temporário.
3. Cadastrar até o limite permitido de números de destino (seu celular) e confirmar o código recebido.
4. Criar um **usuário do sistema** e gerar um **token permanente**; o temporário expira rapidamente e quebra o teste.
5. Subir a API local e abrir o túnel HTTPS; configurar a URL do webhook e o token de verificação no painel; assinar o campo `messages` .
6. Enviar o template de teste `hello_world` para iniciar a conversa; responder do celular para abrir a janela de 24 h.
7. A partir daí, respostas livres funcionam dentro da janela.
8. Para produção: verificar o negócio, registrar número próprio, criar e aprovar templates e cadastrar método de pagamento.

Exemplo de payload de entrada (simplificado):

```
{
  "object": "whatsapp_business_account",
  "entry": [{
    "id": "WABA_ID",
    "changes": [{
      "field": "messages",
      "value": {
        "messaging_product": "whatsapp",
        "metadata": {"phone_number_id": "PHONE_NUMBER_ID"},
        "contacts": [{"profile": {"name": "Maria"}, "wa_id":
"55839XXXXXXXX"}],
        "messages": [{"from": "55839XXXXXXXX", "id": "wamid.XXX",
"timestamp": "1731000000", "type": "text", "text": {"body": "Oi"}}]
      }
    }]
  }]
}
```

Envio de resposta: `POST /{versão}/{phone_number_id}/messages` com cabeçalho `Authorization: Bearer <token>` e corpo `{"messaging_product":"whatsapp","to":"<wa_id>","type":"text","text":{"body":"..."}}`.

Atenção a números brasileiros: o nono dígito pode aparecer de forma diferente entre o número cadastrado e o `wa_id` retornado. Normalizar números com e sem o 9 na comparação e no cadastro de destinatários.

12. Modelo de Dados

contacts: id, wa_id (único), nome, consentimento_marketing, consentimento_em, opt_out_em.

conversations: id, contact_id, estado, janela_expira_em, canal, status (`BOT`, `HUMAN`, `CLOSED`), criada_em.

messages: id, conversation_id, wamid (único), direcao, tipo, conteudo, categoria_cobranca, status_entrega, criada_em.

tickets: id, conversation_id, motivo, prioridade, atendente, status, resumo, criado_em, fechado_em.

appointments: id, contact_id, servico, inicio, fim, status.

knowledge_articles: id, titulo, conteudo, tags, atualizado_em (e embedding na fase 2).

webhook_events: id, payload bruto, wamid, processado_em (auditoria e reprocessamento).

usage_counters: mês, número, categoria, quantidade (controle da cota gratuita).

13. Camada de IA e Guardrails (fase 2)

- **Fase 1 (sem IA):** menus, regras e busca por palavras-chave. Funciona, é barata e previsível.
- **Fase 2:** classificar intenção (FAQ, pedido, agendamento, humano, fora de escopo) e responder FAQ com RAG sobre a base do negócio.
- **Prompt de sistema base:** você é o assistente de atendimento de [empresa]. Responda somente sobre serviços, preços, horários, pedidos e agendamentos, usando apenas o contexto fornecido. Se não houver a informação, diga que não sabe e ofereça um atendente. Recuse outros assuntos de forma educada. Nunca invente preços ou prazos.
- **Proteções:** saída estruturada validada, limite de tamanho, filtro de dados sensíveis, tratamento do texto do usuário como dado (contra injeção de prompt), fallback para humano em baixa confiança.
- **Custos:** usar Ollama local ou o tier gratuito do provedor; abstrair para poder trocar.

14. API Interna (resumo)

Rota	Função
GET /webhook	Verificação da Meta
POST /webhook	Recebimento de eventos
GET /api/v1/conversations	Lista conversas com filtros
POST /api/v1/conversations/{id}/messages	Atendente envia mensagem
POST /api/v1/conversations/{id}/handoff	Transfere para humano ou devolve ao bot
GET /api/v1/metrics	Indicadores de atendimento e consumo
POST /simulator/inbound	Injeta mensagem simulada (somente em ambiente de teste)
GET /health	Saúde do serviço

15. Metodologia

- **Processo:** Scrum leve com sprints de 1 semana e quadro Kanban; cada sprint termina com uma demonstração no celular.
- **Ordem de construção:** primeiro o simulador e a máquina de estados (sem Meta), depois o adaptador Meta, depois a IA.
- **Diálogos dourados:** conjunto de 40 a 80 conversas de teste (roteiros com entrada do cliente e resposta esperada) versionadas no repositório.
- **TDD** para fluxos, validações e regras da janela de 24 h.
- **Contract tests** com payloads reais gravados da Meta (anonimizados).
- **Experimentos de IA:** comparar regras versus classificador versus LLM em taxa de intenção correta, taxa de recusa correta e latência.

16. Estratégia de Testes

Nível	Escopo	Ferramentas
Unitário	Máquina de estados, regras de janela, validação de pedido, normalização de telefone	pytest
Contrato	Parsing de payloads reais e formato das respostas enviadas	pytest + payloads gravados
Diálogo	Roteiros ponta a ponta no simulador	pytest + simulador
Integração	API, fila, banco e Redis reais	docker compose
Sandbox	Teste manual com o número de teste e seu celular	checklist da seção 17
Segurança	Assinatura inválida, payload malformado, repetição de evento, injeção de prompt	pytest
Carga	50 conversas simultâneas simuladas	locust ou script

17. Critérios de Aceite

17.1 Critérios funcionais (Gherkin)

CA-01 Verificação do webhook

- Dado o token de verificação correto
- Quando a Meta chamar `GET /webhook` com `hub.challenge`

- Então o sistema devolve o mesmo desafio com status 200.

CA-02 Assinatura inválida

- Dado um POST com assinatura incorreta
- Então o sistema responde 401/403 e não processa o evento.

CA-03 Deduplicação

- Dado o mesmo `wamid` recebido duas vezes
- Então apenas uma resposta é enviada.

CA-04 Primeiro contato

- Dada uma mensagem inicial de um número novo
- Então o cliente recebe saudação com aviso de atendimento automatizado e menu em menos de 5 s.

CA-05 FAQ

- Dada a pergunta sobre horário de funcionamento
- Então a resposta traz o horário correto da base de conhecimento.

CA-06 Sem resposta na base

- Dada uma pergunta que a base não cobre
- Então o bot informa que não sabe e oferece atendente, sem inventar dados.

CA-07 Status de pedido

- Dado um número de pedido válido
- Então o bot informa o status; com número inválido, pede correção sem consultar.

CA-08 Agendamento

- Dado um horário disponível
- Então o agendamento é criado e confirmado; em horário ocupado, o bot oferece alternativas.

CA-09 Handoff

- Dado o pedido de falar com humano
- Então o ticket é criado com resumo e o bot deixa de responder até a devolução.

CA-10 Janela de 24 h

- Dada uma conversa com última mensagem do cliente há mais de 24 h
- Então o sistema só envia template aprovado e registra a categoria.

CA-11 Opt-out

- Dado o cliente que digita PARAR
- Então o sistema confirma e bloqueia mensagens ativas.

CA-12 Fora de escopo

- Dado um pedido sem relação com o negócio
- Então o bot recusa educadamente e reapresenta o menu.

CA-13 Falha de envio

- Dada uma falha temporária da API
- Então o sistema tenta novamente com backoff e registra o erro, sem perder a mensagem.

17.2 Checklist de teste manual no sandbox

1. Webhook verificado no painel da Meta.
2. Mensagem do seu celular chega ao sistema e aparece no log.
3. Resposta com botões é exibida no celular.
4. Toque no botão retorna evento correto.
5. Mensagem de áudio recebe resposta de tratamento.
6. Handoff cria ticket e o bot silencia.
7. Status de entrega (enviado, entregue, lido) atualiza o banco.
8. Reenvio do mesmo evento (teste de repetição) não gera resposta duplicada.

17.3 Metas de qualidade (diálogos dourados)

Indicador	Mínima	Desejável
Intenção correta	85%	93%
Resolução sem humano	60%	75%
Recusa correta de fora de escopo	90%	97%
Handoff correto	95%	99%
Respostas inventadas	0	0
ACK do webhook (p95)	menor que 1 s	menor que 500 ms
Primeira resposta (p95)	menor que 5 s	menor que 3 s

18. Segurança, Privacidade e Conformidade

- **Webhook:** validar `X-Hub-Signature-256` com o segredo do app; rejeitar o que não confere; limitar tamanho de corpo e taxa.
- **Segredos:** token e segredo do app em variáveis de ambiente; nunca no repositório; rotação periódica.
- **LGPD:** informar finalidade e uso de IA na primeira mensagem, coletar apenas o necessário, registrar consentimento, permitir exclusão, definir retenção (por exemplo, 90 dias) e mascarar telefone em logs.
- **Termos da Meta:** respeitar política de uso comercial, opt-in, qualidade e a regra sobre IA de uso geral (RN-06).
- **Atendimento ao consumidor:** manter acesso a atendente humano e registrar protocolo.
- **Injeção de prompt:** o texto do cliente é dado; sem ferramentas perigosas para a IA; saída validada.
- **Aviso:** este documento não é parecer jurídico; valide LGPD e regras de SAC com um profissional se for atender clientes reais.

19. Custos e Limites

Item	Desenvolvimento/teste	Produção pequena
Cloud API (mensagens de serviço na janela de 24 h)	R$ 0 (número de teste)	1.000 gratuitas por número/mês; depois valor por mensagem conforme tabela oficial
Templates utility/marketing	Limitados no teste	Pagos por mensagem entregue conforme categoria e país
Conversas iniciadas por anúncio click-to-WhatsApp	Não aplicável	Janela de 72 h sem cobrança
Hospedagem	Máquina local + túnel gratuito	VPS pequeno (custo baixo, não gratuito)
IA	Ollama local ou tier gratuito	Depende do volume e do provedor

O sistema deve registrar a categoria de cada mensagem para estimar a fatura (RF-18).

20. Plano de Entrega

Sprint	Entregas	Resultado verificável
0	Repositório, Docker, simulador e `ChannelAdapter`	Mensagem simulada percorre o sistema
1	Máquina de estados, menus, FAQ por regras	CA-04, CA-05, CA-06, CA-12
2	Adaptador Meta, webhook, assinatura, deduplicação, túnel e sandbox	CA-01, CA-02, CA-03 e checklist 17.2
3	Fila, retentativas, status de entrega e janela de 24 h	CA-10, CA-13
4	Pedido, agendamento (mocks) e opt-in/opt-out	CA-07, CA-08, CA-11
5	Handoff, tickets e painel (Chatwoot ou simples)	CA-09
6	IA restrita com RAG, guardrails e avaliação	Metas da seção 17.3
7	Métricas, LGPD, documentação e demonstração	Entrega final

21. Riscos e Mitigações

Risco	Prob.	Impacto	Mitigação
Token temporário expira e o teste para	Alta	Baixo	Token permanente de usuário do sistema
Túnel muda a URL e o webhook para de chegar	Alta	Baixo	Domínio fixo do túnel (cloudflared nomeado) ou script de atualização
Mudança de preços ou políticas da Meta	Alta	Médio	Contador de consumo, configuração por variáveis e revisão das fontes oficiais
Bot tratado como IA de uso geral	Média	Alto	Escopo estrito, recusa de fora de escopo e testes de aderência
Reenvio de eventos causa respostas duplicadas	Média	Médio	Deduplicação por `wamid` com expiração

Risco	Prob.	Impacto	Mitigação
Número restrito por baixa qualidade	Média	Alto	Opt-in claro, opt-out simples, ritmo de envio e monitoramento
Divergência do nono dígito em números brasileiros	Alta	Baixo	Normalização de números
Alucinação da IA sobre preços e prazos	Média	Alto	RAG, validação, recusa e handoff
Dados pessoais em logs	Média	Alto	Mascaramento e retenção limitada

22. Definição de Pronto (DoD)

- Código revisado; lint e tipagem passando no CI.
- Testes unitários, de contrato e de diálogo verdes.
- Checklist do sandbox executado no celular.
- Métricas dos diálogos dourados registradas sem regressão.
- Sem segredos ou dados pessoais no repositório.
- Documentação e variáveis de ambiente atualizadas.

23. Glossário

- **Cloud API:** API oficial hospedada pela Meta para o WhatsApp Business Platform.
- **Webhook:** chamada HTTP enviada pela Meta ao seu servidor quando algo acontece.
- **Janela de 24 h:** período após a última mensagem do cliente em que se pode responder com mensagem livre.
- **Template:** mensagem pré-aprovada pela Meta, necessária fora da janela.
- **wamid:** identificador único de mensagem do WhatsApp.
- **Handoff:** transferência da conversa do bot para uma pessoa.
- **Opt-in/Opt-out:** consentimento para receber mensagens e sua revogação.
- **RAG:** resposta gerada a partir de trechos recuperados de uma base de conhecimento.

24. Alternativa Experimental: Evolution API (não oficial)

A Evolution API, o Baileys e o WPPConnect conectam ao WhatsApp por QR Code, sem custo de API. **Violam os termos da Meta e podem causar banimento do número.**

- **Quando considerar:** apenas para aprendizado com um chip descartável, nunca para clientes nem para o número principal da empresa.
- **Como encaixar:** implementar `evolution.py` como mais um `ChannelAdapter`, mantendo o restante do sistema idêntico.
- **Limitações:** sem garantia de estabilidade, sem templates oficiais, risco de bloqueio, e a janela de 24 h e o opt-in continuam sendo boas práticas.
- **Recomendação:** não usar no portfólio como solução principal; apresentar a Cloud API como padrão e este adaptador apenas como nota técnica.